COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	August 5, 2026	**DATE SUBMITTED:**	August 7, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

ILO A : STL QUEUE

main.cpp

main.cpp > display(std::queue<char>)

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main() {

    // Create an object
    std::queue<char> myQ;

    // Use Enqueue operation
    myQ.push('S');
    myQ.push('H');
    myQ.push('I');
    myQ.push('N');
    myQ.push('J');
    myQ.push('I');

    // Print the front and the back of the queue
    std::cout << "Current front of the queue is: " << myQ.front() << std::endl;
    std::cout << "Current back of the queue is: " << myQ.back() << std::endl;
    std::cout << std::endl;
    // Print the ENTIRE queue
    display(myQ);

    // Use Dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    std::cout << "Is the queue empty? " << myQ.empty() << std::endl;

    return 0;
}
```

```cpp
void display(std::queue<char> copyQ){

    // Create a copy of the queue
    std::queue<char> tempQ(copyQ);
    // Loop until Empty
    while (!tempQ.empty()) {
        std::cout << "Current front of the queue is: " << tempQ.front() << std::endl;
        tempQ.pop();
    }
    std::cout << std::endl;
}
```

Observations: The code demonstrates the use of standard C++ First-In, First-Out (FIFO) queue using std::queue<char> to store and manipulate characters. It populates the queue with the letters of the listed characters, prints the front and back elements, and uses a custom display function to print all elements without modifying the original queue. The program then sequentially pops each element which redisplays the remaining contents after every removal and concludes by confirming the queue is empty.

Output

```
Current front of the queue is: S
Current back of the queue is: I

Current front of the queue is: S
Current front of the queue is: H
Current front of the queue is: I
Current front of the queue is: N
Current front of the queue is: J
Current front of the queue is: I

Current front of the queue is: H
Current front of the queue is: I
Current front of the queue is: N
Current front of the queue is: J
Current front of the queue is: I

Current front of the queue is: I
Current front of the queue is: N
Current front of the queue is: J
Current front of the queue is: I

Current front of the queue is: N
Current front of the queue is: J
Current front of the queue is: I

Current front of the queue is: J
Current front of the queue is: I

Current front of the queue is: I

Is the queue empty? 1
```

Observations: The program demonstrates a C++ queue by sequentially adding the characters and printing the front and back elements. It uses a helper function that copies the queue to print its contents non-destructively, followed by step-by-step element removal. Finally, the code verifies that all elements have been removed by displaying a boolean value confirming the queue is empty.

ILO B1 : QUEUE LINKED LIST

main.cpp

queue.cpp

```cpp
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout << "===== TESTING ENQUEUE =====\n";

    enqueue('E', &front, &back);
    enqueue('Y', &front, &back);
    enqueue('D', &front, &back);
    enqueue('I', &front, &back);
    enqueue('S', &front, &back);

    displayALL(&front, &back);

    std::cout << "Front: " << front->data << std::endl;
    std::cout << "Back : " << back->data << std::endl;

    std::cout << "\nIs Empty? "
              << (isEmpty(&front, &back) ? "Yes" : "No")
              << std::endl;

    std::cout << "\n===== TESTING DEQUEUE =====\n";

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    std::cout << "\nIs Empty? "
              << (isEmpty(&front, &back) ? "Yes" : "No")
              << std::endl;

    // Test dequeue on empty queue
    dequeue(&front, &back);

    return 0;
}
```

queueLL.h

queue.cpp | queueLL.h

queueLL.h > ...

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};

// Create a new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>();
    newNode->data = newData;
    newNode->next = nullptr;

    return newNode;
}

// Enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {

    qNode<T>* newNode = new_node(newData);

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

// Dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout << "The queue is empty." << std::endl;
        return;
    }

    qNode<T>* deleteTemp = *frontPtr;

    // Only one node
    if ((*frontPtr)->next == nullptr) {
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteTemp;
        return;
```

```
    }

    *frontPtr = (*frontPtr)->next;
    delete deleteTemp;
}

// Check if queue is empty
template <typename T>
bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr) {
    return ((*frontPtr) == nullptr && (*backPtr) == nullptr);
}

// Display all elements
template <typename T>
void displayALL(qNode<T>** frontPtr, qNode<T>** backPtr) {

    if (isEmpty(frontPtr, backPtr)) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    qNode<T>* temp = *frontPtr;

    std::cout << "Queue Elements: ";

    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }

    std::cout << std::endl;
}

#endif
```

Output

```
PS C:\Users\mendo\OneDrive\Documents\C++\Queue Linked List\output> & .\'queue.exe'
===== TESTING ENQUEUE =====
Queue Elements: E Y D I S
Front: E
Back : S

Is Empty? No

===== TESTING DEQUEUE =====
Queue Elements: Y D I S
Queue Elements: D I S
Queue Elements: I S
Queue Elements: S
Queue is empty.

Is Empty? Yes
The queue is empty.
PS C:\Users\mendo\OneDrive\Documents\C++\Queue Linked List\output>
```

Observations

Through implementing the queue that is based using a linked list, it helped me visualize how elements are added at the back and removed from the front while maintaining the FIFO (First In, First Out) order. I also learned that testing each operation made it easier to understand how

	the front and back pointers change as the queue becomes full and eventually empty. function call (enqueue, displayALL, isEmpty, and dequeue) demonstrates the queue's current state, while displayALL() lets me see the elements from front to back and isEmpty() confirms whether the queue still contains data. This made it easier to understand how the front and back pointers change after every operation and how each function works together to manage the queue.

ILO B2 : QUEUE ARRAY	
main.cpp	

QueueArrMain.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main()
{
    // creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(3);

    q.enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(5);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(6);

    std::cout << "\nremoved: " << q.dequeue() << std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;

    // testing the clear
    q.Clear();

    std::cout << "is the queue empty? "
              << (q.Empty() ? "yes" : "no")
              << std::endl;

    return 0;
}
```

queueArr.h

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
# include <iostream>

// Array-Based Circular Queue
template <typename T>
class queueArr {
    private:
        // POINTER TO DYNAMICALLY ALLOCATE ARRAY
        T* q_arr;

        // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
        size_t q_capacity;

        // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
        size_t q_size;

        // INDEX OF THE FRONT ELEMENT IN THE QUEUE
        int q_front;

        // INDEX OF THE BACK ELEMENT IN THE QUEUE
        int q_back;
```

```h
    public:

    // CONSTRUCTOR
    queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
    }

    // COPY CONSTRUCTOR
    queueArr(const queueArr& other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
        // COPY ELEMENTS FROM OTHER QUEUE
        for (size_t i = 0; i < q_size; ++i) {
            q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
        }
    }

    // COPY ASSIGNMENT OPERATOR
    queueArr& operator=(const queueArr& other) {
        if (this != &other) {
            // DEALLOCATE CURRENT ARRAY
            delete[] q_arr;

            // COPY CAPACITY AND SIZE
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;

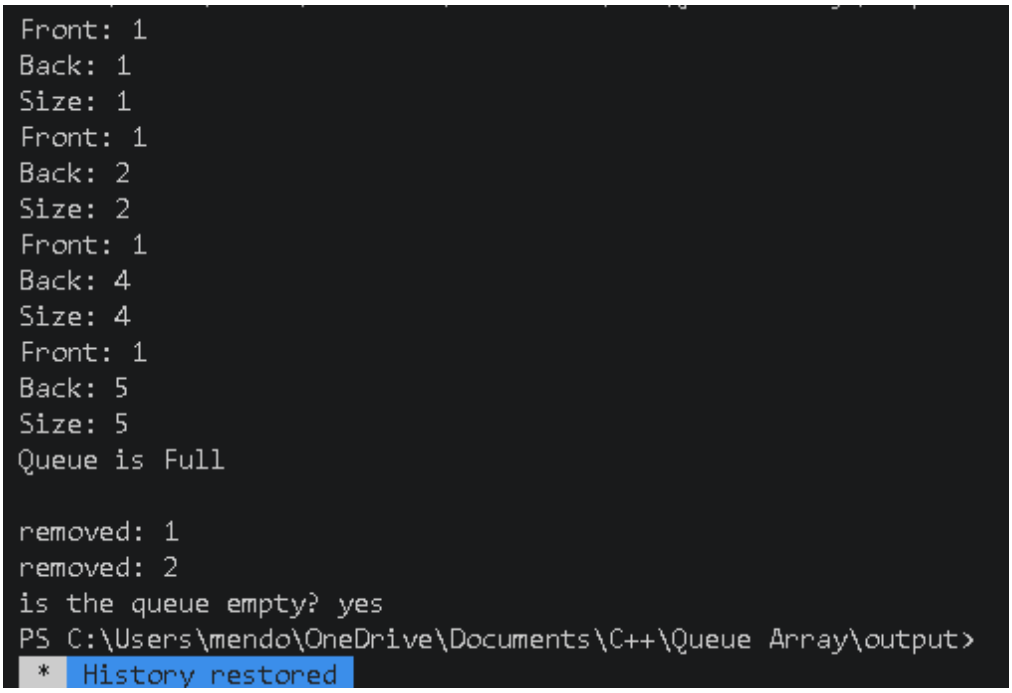
            // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
            q_arr = new T[q_capacity];
            // COPY ELEMENTS FROM OTHER QUEUE
            for (size_t i = 0; i < q_size; ++i) {
                q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
            }
        }
        return *this;
    }

    // DESTRUCTOR
    ~queueArr() {
        // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
        delete[] q_arr;
    }
```

```
70      // QUEUE OPERATIONS
71      bool Empty();
72      bool Full();
73      size_t Size();
74      void Clear();
75      T Front();
76      T Back();
77      void enqueue(T value);
78      T dequeue();
79  };
```

```
81  // Empty()
82  template <typename T>
83  bool queueArr<T>::Empty() {
84      // CHECK IF EMPTY
85      return q_size == 0;
86  }
87
88  // Full()
89  template <typename T>
90  bool queueArr<T>::Full() {
91      // CHECK IF REACHES MAX CAPACITY
92      return q_size == q_capacity;
93  }
94
95  // Size()
96  template <typename T>
97  size_t queueArr<T>::Size() {
98      // RETURN CURRENT SIZE OF QUEUE
99      return q_size;
100 }
101
102 // Front()
103 template <typename T>
104 T queueArr<T>::Front() {
105     // Check if the queue is empty
106     if (Empty()) {
107         std::cout << "Queue is Empty\n";
108         return T(); // Return default value of type T
109     }
110     // Return the front
111     return q_arr[q_front];
112 }
113
114 // Back()
115 template <typename T>
116 T queueArr<T>::Back() {
117     // Check if the queue is empty
118     if (Empty()) {
119         std::cout << "Queue is Empty\n";
120         return T(); // Return default value of type T
121     }
122     // Return the back
123     return q_arr[q_back];
124 }
125
```

```
126  // Clear()
127  template <typename T>
128  void queueArr<T>::Clear() {
129      // Reset the queue to its initial state
130      q_size = 0;
131      q_front = 0;
132      q_back = -1; // Set back to -1 since there are no e
133  }
134
135  // Enqueue()
136  template <typename T>
137  void queueArr<T>::enqueue(T value) {
138      // Check if the queue is full
139      if (Full()) {
140          std::cout << "Queue is Full\n";
141          return;
142      }
143      // Increment the back
144      q_back = (q_back + 1) % q_capacity;
145      // Add the new value to the back of the queue
146      q_arr[q_back] = value;
147      // Increment the size of the queue
148      q_size++;
149  }
150
151  // Dequeue()
152  template <typename T>
153  T queueArr<T>::dequeue() {
154      // Check if the queue is empty
155      if (Empty()) {
156          std::cout << "Queue is Empty\n";
157          return T(); // Return default value of type T
158      }
159      // Store the front value
160      T value = q_arr[q_front];
161      // Increment the front index in a circular manner
162      q_front = (q_front + 1) % q_capacity;
163      // Decrement the size of the queue
164      q_size--;
165      // Reset the indexes
166      if (Empty()) Clear();
167      // Return the dequeued value
168      return value;
169  }
170
171  #endif // QUEUEARR_H
```

Output	
Observations	The copy constructor and copy assignment operator visualized how a queue creates its own separate copy instead of sharing the same memory. This means that the copy constructor allocates a new array and copies each element into it, while the copy assignment operator first deletes the old array, leaving that memory empty, before allocating a new one and copying the updated data. This made it easier to see how memory is safely managed and why each queue keeps its own independent copy without affecting the other.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C : Solve Problems Using Queue Implementations	
HEADER FILES	
Job.h	

Job.h

```h
#ifndef JOB_H
#define JOB_H

#include <string>
#include <iostream>

class Job {
    private:
        std::string Username;
        int jobID;
        int paperCount;

    public:
        // Default Constructor
        Job(){
            Username = "";
            jobID = 0;
            paperCount = 0;
        }

        // Constructor
        Job(std::string user, int id, int count){
            Username = user;
            jobID = id;
            paperCount = count;
        }

        // Getters
        int getJobID() const {
            return jobID;
        }

        std::string getUsername() const {
            return Username;
        }

        int getPaperCount() const {
            return paperCount;
        }

        // Display function
        void display() const {
            std::cout << "Job ID: " << jobID
                      << ", Username: " << Username
                      << ", Paper Count: " << paperCount;
        }

        // Stream insertion operator overload
        friend std::ostream& operator<<(std::ostream& os, const Job& job) {
            os << "Job ID: " << job.jobID
               << ", Username: " << job.Username
               << ", Paper Count: " << job.paperCount;
            return os;
        }
};

#endif // JOB_H
```

	Observations: This header defines a lightweight Job class that stores print request attributes like username, ID, and paper count. It handles basic data representation and output stream formatting without complex logic. When a user visualizes a printer queue, this file defines the individual print tasks occupying memory.
Printer.h	

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <string>
#include "Job.h"
#include "queueArr.h"

class Printer {
private:
    queueArr<Job> jobQueue;

public:
    Printer(int capacity) : jobQueue(capacity) {}

    void addJob(const Job& job) {
        jobQueue.enqueue(job);
        std::cout << "\nJob Added Successfully!\n";
        job.display();
        std::cout << "\n";
    }

    void displayAndRemoveJob() {
        if (jobQueue.Empty()) {
            std::cout << "\nNo jobs available to display or remove.\n";
            return;
        }

        // 1. Display current elements with index positions
        jobQueue.displayQueue();

        // 2. Ask user which index to remove or to exit
        int indexToRemove;
        std::cout << "\nEnter array index position to remove (or -1 to Return/Exit): ";
        std::cin >> indexToRemove;

        // Check for exit choice
        if (indexToRemove == -1) {
            std::cout << "\nReturning to main menu without removing any jobs.\n";
            return;
        }

        // 3. Remove selected index
        if (indexToRemove >= 0 && jobQueue.removeAt(static_cast<size_t>(indexToRemove))) {
            std::cout << "\nSuccessfully removed element at index " << indexToRemove << "!\n";

            // 4. Display updated array state
```

```
            // 4. Display updated array state
            std::cout << "\n===== Updated Queue State =====\n";
            jobQueue.displayQueue();
        }
    }

    void runProgram() {
        int choice;
        std::string user;
        int id;
        int pages;

        do {
            std::cout << "\n====================================\n";
            std::cout << "      SHARED PRINTER SIMULATOR\n";
            std::cout << "====================================\n";
            std::cout << "1. Add Print Job\n";
            std::cout << "2. Display Jobs & Remove Specific Index\n";
            std::cout << "3. Exit\n";
            std::cout << "====================================\n";
            std::cout << "Enter Choice: ";
            std::cin >> choice;

            switch (choice) {
            case 1: {
                std::cin.ignore();
                std::cout << "\nEnter Username: ";
                getline(std::cin, user);

                std::cout << "Enter Job ID: ";
                std::cin >> id;

                std::cout << "Enter Number of Pages: ";
                std::cin >> pages;

                addJob(Job(user, id, pages));
                break;
            }

            case 2: {
                displayAndRemoveJob();
                break;
                }

                case 3: {
                    std::cout << "\nExiting Program...\n";
                    break;
                }

                default: {
                    std::cout << "\nInvalid Choice!\n";
                }
                }
            } while (choice != 3);
        }
    };

    #endif
```

	Observations: This header acts as the system controller, bridging user inputs to the underlying queueArr storage. It simplifies interaction by presenting a streamlined menu for adding, displaying, and selectively deleting print jobs. Human interaction happens here, as the program translates human choices into structured data structure operations.
queueArr.h	

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

template <typename T>
class queueArr {
private:
    T* q_arr;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    queueArr(size_t capacity) : q_size(0), q_front(0), q_back(-1) {
        if (capacity == 0) capacity = 1;
        q_capacity = capacity;
        q_arr = new T[q_capacity];
    }

    queueArr(const queueArr& other)
        : q_capacity(other.q_capacity), q_size(other.q_size),
          q_front(other.q_front), q_back(other.q_back) {
        q_arr = new T[q_capacity];
        for (size_t i = 0; i < q_capacity; i++) {
            q_arr[i] = other.q_arr[i];
        }
    }

    queueArr& operator=(const queueArr& other) {
        if (this != &other) {
            delete[] q_arr;
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;
            q_arr = new T[q_capacity];
            for (size_t i = 0; i < q_capacity; i++) {
                q_arr[i] = other.q_arr[i];
            }
        }
        return *this;
    }

    ~queueArr() {
        delete[] q_arr;
    }
```

```
    bool Empty() const { return q_size == 0; }
    bool Full() const { return q_size == q_capacity; }
    size_t Size() const { return q_size; }

    void Clear() {
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    void enqueue(const T& value) {
        if (Full()) {
            std::cout << "Queue is Full\n";
            return;
        }
        q_back = (q_back + 1) % q_capacity;
        q_arr[q_back] = value;
        q_size++;
    }

    T dequeue() {
        if (Empty()) {
            std::cout << "Queue is Empty\n";
            return T();
        }
        T value = q_arr[q_front];
        q_front = (q_front + 1) % q_capacity;
        q_size--;
        if (Empty()) Clear();
        return value;
    }

    // Remove element at a specific offset/position relative to q_front
    bool removeAt(size_t index) {
        if (index >= q_size) {
            std::cout << "Invalid index position!\n";
            return false;
        }

        // Shift elements down to fill the removed position's slot
        for (size_t i = index; i < q_size - 1; i++) {
            int currentIdx = (q_front + i) % q_capacity;
            int nextIdx = (q_front + i + 1) % q_capacity;
            q_arr[currentIdx] = q_arr[nextIdx];
        }

        // Adjust back pointer and queue size
        q_back = (q_back - 1 + q_capacity) % q_capacity;
        q_size--;

        if (Empty()) Clear();
        return true;
    }

    void displayQueue() const {
        if (Empty()) {
            std::cout << "Queue is Empty\n";
            return;
        }

        std::cout << "\n===== Queue Contents =====\n";
        for (size_t i = 0; i < q_size; i++) {
            T current = q_arr[(q_front + i) % q_capacity];
            std::cout << "Index [" << i << "]: " << current << "\n";
            std::cout << "----------------------------\n";
        }
    }
};
#endif
```

	Observations: This template file implements an array-based circular queue with custom middle-element deletion (removeAt). Unlike standard FIFO queues that only modify boundaries, manual array deletion forces remaining elements to shift left to maintain order. This approach provides continuous memory allocation but trades off constant-time performance for manual index management.
MAIN DRIVER	
main.cpp	

```cpp
#include "Printer.h"
int main()
{
    Printer PaperPrinter(7);
    PaperPrinter.runProgram();
    return 0;
}
```

Observations:This file serves strictly as the application entry point to kick off program execution. It initializes a single Printer object with a fixed array capacity of seven items and transfers control to the menu loop. It acts as the driver, setting up runtime boundaries before user input takes over.

OUTPUT

```
====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 1

Enter Username: Eydis
Enter Job ID: 1
Enter Number of Pages: 67

Job Added Successfully!
Job ID: 1, Username: Eydis, Paper Count: 67

====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 1

Enter Username: Zhongli
Enter Job ID: 2
Enter Number of Pages: 56

Job Added Successfully!
Job ID: 2, Username: Zhongli, Paper Count: 56
```

```
====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 1

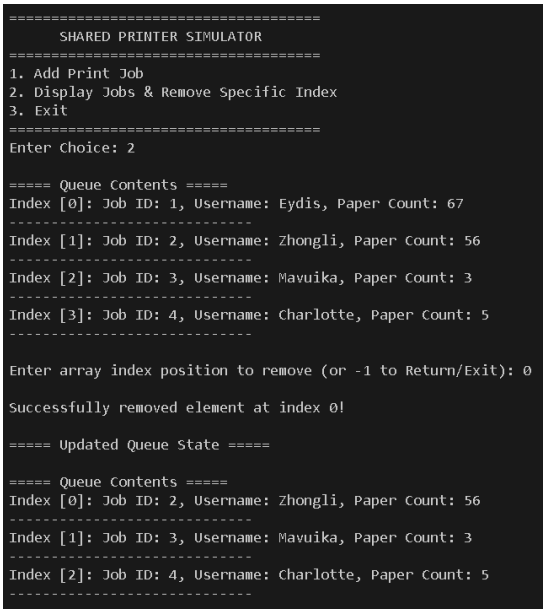
Enter Username: Mavuika
Enter Job ID: 3
Enter Number of Pages: 3

Job Added Successfully!
Job ID: 3, Username: Mavuika, Paper Count: 3

====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 1

Enter Username: Charlotte
Enter Job ID: 4
Enter Number of Pages: 5

Job Added Successfully!
Job ID: 4, Username: Charlotte, Paper Count: 5
```



```
====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 2

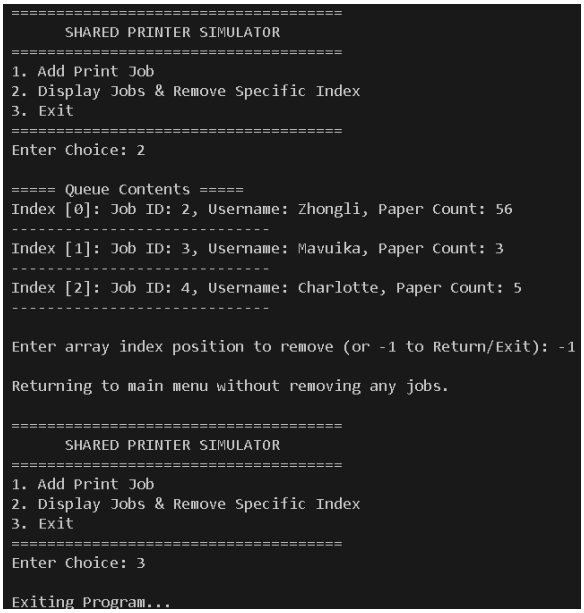
===== Queue Contents =====
Index [0]: Job ID: 1, Username: Eydis, Paper Count: 67
----------------------------
Index [1]: Job ID: 2, Username: Zhongli, Paper Count: 56
----------------------------
Index [2]: Job ID: 3, Username: Mavuika, Paper Count: 3
----------------------------
Index [3]: Job ID: 4, Username: Charlotte, Paper Count: 5
----------------------------

Enter array index position to remove (or -1 to Return/Exit): 0

Successfully removed element at index 0!

===== Updated Queue State =====

===== Queue Contents =====
Index [0]: Job ID: 2, Username: Zhongli, Paper Count: 56
----------------------------
Index [1]: Job ID: 3, Username: Mavuika, Paper Count: 3
----------------------------
Index [2]: Job ID: 4, Username: Charlotte, Paper Count: 5
----------------------------
```



```
====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 2

===== Queue Contents =====
Index [0]: Job ID: 2, Username: Zhongli, Paper Count: 56
----------------------------
Index [1]: Job ID: 3, Username: Mavuika, Paper Count: 3
----------------------------
Index [2]: Job ID: 4, Username: Charlotte, Paper Count: 5
----------------------------

Enter array index position to remove (or -1 to Return/Exit): -1

Returning to main menu without removing any jobs.

====================================
      SHARED PRINTER SIMULATOR
====================================
1. Add Print Job
2. Display Jobs & Remove Specific Index
3. Exit
====================================
Enter Choice: 3

Exiting Program...
```

OBSERVATIONS

The output demonstrates the transition from standard queue operations to direct array manipulation. Enqueueing adds jobs sequentially, but choosing to remove a specific index triggers an array shift where remaining elements slide forward to fill the open slot..

Why Queue Array?

Using an array-based queue made it easier for me to see how elements are stored in consecutive memory locations. I noticed that adding and removing items is fast as long as they follow the normal FIFO order, but removing an item from the middle requires the remaining elements to shift. This helped me understand why array-based queues are efficient for regular queue operations but less practical when frequent middle deletions are needed. Additionally, I implemented these in addition for my learnings as doing today's activity was quite difficult.

CONCLUSION

This activity helped me better understand how a circular array queue works by simulating a real-world printer queue. Because of these I was able to see how jobs were added, processed, and removed and made it easier to visualize how queue operations work in actual applications. Overall, the exercise improved my understanding of queue implementation and showed me the strengths and limitations of using an array-based queue.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.